

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

CSCE 605 — Compiler Design

Front end, intermediate representation, optimisation, and code generation — with the shader pipeline as the worked example

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 5 (with CSCE 620 and CSCE 678)

Prerequisites: CSCE 629 Analysis of Algorithms; CSCE 614 Computer Architecture; fluency in C, C++, or Rust

Effort: 12–14 hours per week · 13 modules + 2 project weeks

Deliverable: A working compiler for a small language — lexer through register allocation — with an SSA intermediate representation, a measured optimisation pipeline, and a differential test suite that proves it does not miscompile

Course description

A compiler is the program every other program depends on, and it is the one whose failures are hardest to see. **A miscompile is invisible from the source.** The code is right, the test is right, the output is wrong, and nothing in the program you wrote explains it — which is why compiler bugs cost so much more than their frequency suggests, and why this course treats correctness as the primary concern rather than a later one.

The organising decision in any compiler is **the intermediate representation**. Everything upstream exists to produce it and everything downstream consumes it, so the IR determines which analyses are cheap, which optimisations are expressible, and which are effectively impossible. **Module 07 argues that static single assignment is the single most consequential idea in modern compilation** — not because it is clever, but because it makes the questions optimisation needs to ask into questions with $O(1)$ answers.

The second theme is that **optimisation is a search over a space with no reliable gradient**. Passes interact, the order matters, a pass that usually helps sometimes hurts, and the only honest way to settle any of it is measurement — which is CSCE 735's discipline applied to the compiler rather than to the program.

The worked example throughout is **the shader pipeline**, because it is the compiler every graphics programmer already depends on and almost none can describe. HLSL and GLSL compile to SPIR-V, SPIR-V is consumed by a driver compiler, and the driver generates machine code for hardware whose instruction set is undocumented. **Module 13 explains shader compilation stutter properly**, which is worth the module on its own.

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 Explain the phases of a compiler and what each establishes.
- 2 Build a lexer from regular expressions via NFA and DFA construction.
- 3 Write recursive-descent and operator-precedence parsers.
- 4 Explain LR parsing and diagnose shift-reduce and reduce-reduce conflicts.
- 5 Implement scope resolution and type checking.
- 6 Construct an SSA intermediate representation with dominance frontiers.
- 7 Formulate and solve dataflow problems as fixpoints over a lattice.
- 8 Implement the standard optimisations and measure which ones pay.
- 9 Perform instruction selection, scheduling, and register allocation.
- 10 Explain shader compilation, pipeline state objects, and why games stutter.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	What a Compiler Is	What does a compiler actually do, and in what order?
2	Lexical Analysis	How do you split text into the words of a language?
3	Parsing I: Top-Down	How do tokens become a tree?
4	Parsing II: Bottom-Up	What can a parser generator do that you cannot by hand?
5	Names, Scopes, and Symbol Tables	What does this name refer to, here?
6	Type Checking	What can a type system establish, and what does it cost?
7	Intermediate Representation and SSA	What shape should the program have for optimisation?
8	Dataflow Analysis	What is true here, on every path that reaches here?
9	Optimisation	Which transformations actually make programs faster?
10	Code Generation	How do you choose instructions, and in what order?
11	Register Allocation	Which values live in registers, and which spill?
12	Runtimes: Garbage Collection, JIT, and Scripting	What does a language need at runtime, and what does it cost?

Wk	Module	Core question it answers
13	Shader Compilation, Testing, and Shipping	Why do games stutter, and how do you test a compiler?

Projects

Project 1 — Front end to IR (after Module 7)

A complete front end for a small imperative language, ending in a well-formed SSA intermediate representation that you can print and inspect.

Requirements

- A lexer, hand-written or generated, with **accurate source positions carried through every later phase**.
- A parser producing an AST, with **error recovery that reports more than one error per run**.
- Scope resolution and a symbol table handling shadowing, nested functions, and forward references.
- A type checker with **diagnostics that name the source location and explain the mismatch**.
- **SSA construction with correctly placed ϕ functions**, computed from dominance frontiers rather than guessed.
- A readable textual IR dump, and an IR verifier.

Done means

- **The IR verifier passes on every test program**, and the verifier checks dominance, not merely well-formedness.
- **A diagnostics gallery**: twenty ill-formed programs with the message each produces. **Quality of diagnostics is graded** — it is most of what users experience.
- An interpreter over your IR, so correctness is testable before any code generation exists.
- **A statement of what your language cannot express** and what that bought you.

Project 2 — Optimisation and code generation (after Module 12)

Take the IR to machine code, with every optimisation measured individually — including the ones that lose.

Requirements

- **At least four optimisation passes** implemented over SSA: constant propagation, dead code elimination, common subexpression elimination or GVN, and one loop transformation.
- Instruction selection and a working backend for one real target, or emission of LLVM IR with a justification.
- **Register allocation by graph colouring or linear scan**, with spilling that works.
- **A differential test suite against an interpreter or another compiler**, run on generated programs.
- A benchmark suite with each pass's effect measured separately.

Done means

- **A table of every pass with its measured effect on each benchmark, including the negative results.** A pass that never helps is a finding, not a failure.
- **Evidence about pass ordering:** at least two orderings measured, with the difference explained.
- **A miscompile found and fixed,** with the reduced test case. If differential testing found none, say how many programs were tested — that is the actual claim.
- An honest comparison against gcc -O0 and -O2, and a statement of where the gap is.

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
Stanford CS143 — Compilers (free, self-paced)	The primary source for Modules 02 through 06. Complete lectures, assignments, and a full compiler project. Free and well paced.
Bob Nystrom — Crafting Interpreters (free book)	The best free introduction in existence. Two complete implementations, every line explained. Read the whole thing early — it makes the rest of the course concrete.
Cooper & Torczon — Engineering a Compiler	The reference for Modules 07 through 11. Stronger than the Dragon Book on SSA, dataflow, and register allocation. Library copy.
LLVM — the Kaleidoscope tutorial and the Language Reference (free)	A working front end in a weekend, and then the IR specification that Module 07 is about. The Language Reference is worth reading as a document in its own right.
Khronos — the SPIR-V specification and SPIRV-Cross / DXC (free)	The Module 13 material. SPIR-V is an SSA IR you can read, which makes it an unusually good teaching artifact as well as a shipping one.

Resource	Role
Muchnick — Advanced Compiler Design and Implementation	The exhaustive reference for Modules 08 and 09. Dense; use it to look things up rather than to read through.

Tooling

- **A systems language** — C++17, Rust, or OCaml. You will write a great deal of tree and graph manipulation, and the language should make tagged unions pleasant.
- **No parser generator for Module 03.** Write the recursive-descent parser by hand; **use a generator in Module 04**, after you understand what it is doing.
- **LLVM installed**, for Modules 07 and 09. `opt -print-after-all` is the single most instructive command in this course.
- **A SPIR-V toolchain** — `glslangValidator` or `DXC`, plus `spirv-dis` and `spirv-opt`. All free, and Module 13 needs them.
- **A differential test harness and a random program generator.** Module 13 argues this is the only testing that finds miscompiles, and `Csmith` is the model.
- **A benchmark suite you can run in one command.** Every claim in Modules 09 through 11 is a measurement, and retuning must be cheap.

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.